

Bracket Pairs

Problem Description

When writing code or parsing mathematical expressions, we often need to check whether brackets `()`, `[]`, `{}` are correctly matched. In this assignment, you will implement this check using an **array-based Stack**, and additionally report several statistics about the bracket structure. Given a string (containing only English letters, digits, and the six bracket characters `( ) [ ] { }`, length ≤ 1000), determine whether the brackets are validly matched. A string is considered valid if and only if:

1. Every closing bracket matches the most recent unmatched opening bracket of the **same type**.
2. Every opening bracket eventually has a matching closing bracket (i.e., no unmatched opening brackets remain).

Input Format

Line 1: An integer T , the number of test cases ($1 \leq T \leq 20$).

Next T lines: Each line contains one string S , the bracket sequence to be checked.

- Length: $0 \leq |S| \leq 1000$ (**empty strings are allowed** — a line may contain no characters at all).
- Character set: only English letters (upper- or lower-case), digits **0–9**, and the six bracket characters `( ) [ ] { }`.
- No whitespace characters (space, tab) appear in the string.
- Each line ends with a newline character.

Output Format

Case 1: Invalid

If the bracket sequence is **not** validly matched, output exactly one line:

Invalid

Case 2: Valid

If the bracket sequence is validly matched, output exactly two lines:

```
Valid
Total pairs = P
```

- **P**: the total number of successfully matched bracket pairs. If the string contains no brackets at all, $P = 0$.

Constraints

- $1 \leq T \leq 20$ °
 - $0 \leq |S| \leq 1000$ °
1. You **must** implement the stack using an **array**. Linked lists or standard library container types are not allowed — the point of this assignment is to practice array-based stack implementation.
 2. You must implement the following functions yourself (exact types and stack capacity are up to you):

```
void push(char c);
char pop(void);
char top(void);
int isEmpty(void);
```

Example Test Case

Sample Input 1

```
2
a(b(c
a(b[c]d)e
```

Sample Input 2

```
2
(a[b{e}])
[()]
```

Sample Output 1

```
Invalid
Valid
Total pairs = 2
```

Sample Output 2

```
Valid
Total pairs = 3
Invalid
```